

SQL Assignment

Q1. Show all the customers whose creditLimit is greater than 20,000.

Approach: We use a simple SELECT with a WHERE clause to filter the customers whose creditLimit is greater than 20,000 using the greater-than (>) operator. We only select the customerNumber, customerName, and creditLimit columns for clarity.

```
SELECT customerNumber, customerName, creditLimit
FROM customers
WHERE creditLimit > 20000;
```

Q2. Show the employees who report to VP Sales.

Approach: We use a subquery inside the WHERE clause to first find the employeeNumber of the employee whose jobTitle is 'VP Sales'. Then in the outer query, we filter all employees whose reportsTo value matches that employeeNumber (of 'VP Sales').

```
SELECT *
FROM employees
WHERE reportsTo = (
    SELECT employeeNumber
    FROM employees
    WHERE jobTitle = 'VP Sales'
);
```

Q3. Find all the customers who have set their state while filling the forms and live in USA and have a credit limit between 100,000 and 200,000.

Approach: We use three conditions combined with AND in the WHERE clause. We check that state IS NOT NULL to confirm the field was filled, we filter country = 'USA' for location, and we use BETWEEN to check that the creditLimit falls within 100,000 and 200,000 (inclusive).

```
SELECT *
FROM customers
WHERE state IS NOT NULL
```

```
AND country = 'USA'  
AND creditLimit BETWEEN 100000 AND 200000;
```

Q4. Find all the employees who report to Sales Managers of all types.

Approach: We use a subquery with the IN operator to collect all employeeNumbers whose jobTitle contains 'Sales Manager' using the LIKE operator with % wildcards on both sides. Then the outer query fetches all employees whose reportsTo value is in that list.

```
SELECT *  
FROM employees  
WHERE reportsTo IN (  
    SELECT employeeNumber  
    FROM employees  
    WHERE jobTitle LIKE '%Sales Manager%'  
);
```

Q5. Find the average credit limit of customers of each country.

Approach: We use GROUP BY on the country column to group customers by their country and then apply the AVG() aggregate function on creditLimit. Each row in the result shows one country along with the average credit limit of its customers.

```
SELECT country, AVG(creditLimit) AS avg_credit_limit  
FROM customers  
GROUP BY country;
```

Q6. Find the total number of orders for each date and customer. Show only those groups where total orders are greater than 10.

Approach: We group the orders by both orderDate and customerNumber and use COUNT(*) to count the total orders in each group. We then use HAVING instead of WHERE to filter after aggregation, keeping only those groups where the count is greater than 10.

```
SELECT orderDate, customerNumber, COUNT(*) AS total_orders  
FROM orders
```

```
GROUP BY orderDate, customerNumber
HAVING COUNT(*) > 10;
```

Q7. Find the name of the supervisor, their job title, and total number of supervisees — using a subquery (without JOIN).

Approach: We self-reference the employees table by treating each row as a potential supervisor. For each supervisor, we use a correlated scalar subquery to count how many employees have their reportsTo set to that supervisor's employeeNumber. We use EXISTS in the outer WHERE clause to make sure we only return employees who actually have at least one supervisee. EXISTS is more efficient than IN because it stops searching as soon as a match is found.

```
SELECT
    supervisor.firstName,
    supervisor.lastName,
    supervisor.jobTitle,
    (
        SELECT COUNT(*)
        FROM employees e
        WHERE e.reportsTo = supervisor.employeeNumber
    ) AS total_supervisees
FROM employees supervisor
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.reportsTo = supervisor.employeeNumber
);
```

Q8. Find the name of the supervisor, their job title, and total number of supervisees — using JOIN.

Approach: We self-join the employees table by using it twice with different aliases; 'e' for the supervisee and 's' for the supervisor. We join them on the condition e.reportsTo = s.employeeNumber. Then we GROUP BY the supervisor's details and use COUNT() to count how many employees fall under each supervisor.

```
SELECT
    s.firstName,
    s.lastName,
    s.jobTitle,
    COUNT(e.employeeNumber) AS total_supervisees
FROM employees e
```

```

JOIN employees s
  ON e.reportsTo = s.employeeNumber
GROUP BY
  s.employeeNumber,
  s.firstName,
  s.lastName,
  s.jobTitle;

```

Q9. Find all customers with a credit limit greater than the average credit limit — using the WITH clause.

Approach: We use the WITH clause to define a CTE named avg_credit that computes the average creditLimit of all customers just once. Then in the main query, we reference this CTE using a cross join and filter only those customers whose creditLimit is greater than the stored average value.

```

WITH avg_credit AS (
  SELECT AVG(creditLimit) AS avg_limit
  FROM customers
)
SELECT
  customerNumber,
  customerName,
  creditLimit
FROM customers, avg_credit
WHERE creditLimit > avg_credit.avg_limit;

```

Q10. Rank customers by credit limit (highest = rank 1). Then find the customer with the third highest credit limit.

Approach: We use the RANK() window function with ORDER BY creditLimit DESC inside the OVER() clause to assign ranks to all customers without collapsing rows. We wrap this in a subquery so we can then filter using WHERE credit_rank = 3 to get the customer with the third highest credit limit.

```

SELECT *
FROM (
  SELECT
    customerNumber,
    customerName,
    creditLimit,
    RANK() OVER (ORDER BY creditLimit DESC) AS credit_rank
  FROM customers
)

```

```
) ranked_customers  
WHERE credit_rank = 3;
```

Q11. Generate a report showing the total number of employees working in each office.

Approach: We JOIN the offices and employees tables on the officeCode column. Then we GROUP BY the office details and use COUNT(employeeNumber) to count how many employees work in each office along with the office location.

```
SELECT  
    o.officeCode,  
    o.city,  
    o.country,  
    COUNT(e.employeeNumber) AS total_employees  
FROM offices o  
JOIN employees e  
    ON o.officeCode = e.officeCode  
GROUP BY  
    o.officeCode, o.city, o.country;
```

Q12. Generate a report showing the total number of customers served by each office.

Approach: We chain three JOINS together; first we join customers to employees using salesRepEmployeeNumber, and then we join employees to offices using officeCode. This traces each customer back to the office that serves them. We then GROUP BY office and use COUNT to find the total customers per office.

```
SELECT  
    o.officeCode,  
    o.city,  
    o.country,  
    COUNT(c.customerNumber) AS total_customers  
FROM customers c  
JOIN employees e  
    ON c.salesRepEmployeeNumber = e.employeeNumber  
JOIN offices o  
    ON e.officeCode = o.officeCode  
GROUP BY  
    o.officeCode, o.city, o.country;
```

Q13. Generate a report showing total payments received by each office — including city, state, country, and total amount.

Approach: We join four tables; payments to customers, customers to employees, and employees to offices to trace each payment back to its office. We then use SUM(p.amount) to get the total payments per office and GROUP BY the office details to structure the report.

```
SELECT
    o.officeCode,
    o.city,
    o.state,
    o.country,
    SUM(p.amount) AS total_payments
FROM payments p
JOIN customers c ON p.customerNumber = c.customerNumber
JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber
JOIN offices o ON e.officeCode = o.officeCode
GROUP BY
    o.officeCode, o.city, o.state, o.country;
```

Q14. Generate a report showing total sales amount by each office using the order details table.

Approach: We join orderdetails to orders, then to customers, employees, and finally offices. We calculate the total sales as SUM(quantityOrdered * priceEach) which gives us the actual billed amount rather than payments received. We GROUP BY office to get the total per office.

```
SELECT
    o.officeCode,
    o.city,
    o.country,
    SUM(od.quantityOrdered * od.priceEach) AS total_sales
FROM orderdetails od
JOIN orders ord ON od.orderNumber = ord.orderNumber
JOIN customers c ON ord.customerNumber = c.customerNumber
JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber
JOIN offices o ON e.officeCode = o.officeCode
GROUP BY
    o.officeCode, o.city, o.country;
```

Q15. Generate a report showing total pending payment for each office.

Approach: We use two CTEs; 'sales' to compute the total billed amount per office using quantityOrdered * priceEach, and 'payments' to compute the total amount received per office. We then LEFT JOIN the two CTEs and subtract total_payments from total_sales to get the pending amount. We use COALESCE to handle offices that have no payments recorded so they don't result in NULL.

```
WITH sales AS (  
    SELECT o.officeCode, o.city, o.country,  
           SUM(od.quantityOrdered * od.priceEach) AS total_sales  
    FROM orderdetails od  
    JOIN orders ord ON od.orderNumber = ord.orderNumber  
    JOIN customers c ON ord.customerNumber = c.customerNumber  
    JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber  
    JOIN offices o ON e.officeCode = o.officeCode  
    GROUP BY o.officeCode, o.city, o.country  
)  
,  
payments AS (  
    SELECT o.officeCode, SUM(p.amount) AS total_payments  
    FROM payments p  
    JOIN customers c ON p.customerNumber = c.customerNumber  
    JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber  
    JOIN offices o ON e.officeCode = o.officeCode  
    GROUP BY o.officeCode  
)  
SELECT s.officeCode, s.city, s.country,  
       s.total_sales - COALESCE(p.total_payments, 0) AS pending_amount  
FROM sales s  
LEFT JOIN payments p ON s.officeCode = p.officeCode;
```

Q16. Find the credit limit of each customer and their proportion of the total credit limit within their country.

Approach: We use SUM(creditLimit) OVER (PARTITION BY country) as a window function to calculate the total creditLimit for each country without collapsing the rows. We then divide each customer's individual creditLimit by this country total to get their proportion within the country.

```
SELECT  
    customerNumber,  
    customerName,  
    country,  
    creditLimit,  
    creditLimit / SUM(creditLimit) OVER (PARTITION BY country) AS proportion  
FROM customers;
```

Q17. Create a view showing customer name, complete address, and their total number of orders.

Approach: We use CREATE VIEW to save this query as a reusable database object. We use CONCAT to build the full address from individual address columns. We use LEFT JOIN with orders so that customers who have placed no orders are still included in the result. We GROUP BY customerNumber to avoid duplicate rows per customer.

```
CREATE VIEW customer_order_summary_view AS
SELECT
    c.customerName,
    CONCAT(c.addressLine1, ', ', c.city, ', ', c.country) AS full_address,
    COUNT(o.orderNumber) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customerNumber = o.customerNumber
GROUP BY
    c.customerNumber, c.customerName,
    c.addressLine1, c.city, c.country;

SELECT * FROM customer_order_summary_view;
```

Q18. Update the country of a customer (use any one record).

Approach: We use the UPDATE statement with SET to change the country field to 'Nepal'. We use the WHERE clause with a specific customerNumber (103) to make sure only that one record gets updated and we do not accidentally modify other rows.

```
UPDATE customers
SET country = 'Nepal'
WHERE customerNumber = 103;
```

Q19. Delete all payments below 20,000.

Approach: We use the DELETE statement with a WHERE clause that filters rows where amount is less than 20,000. This permanently removes only those payment records that meet the condition while keeping all other rows in the table intact.

```
DELETE FROM payments
WHERE amount < 20000;
```


Q20. Add a new payment manually for an existing customer.

Approach: We use INSERT INTO with explicit column names followed by a VALUES clause to add a new payment record. We provide all the required fields; an existing customerNumber, a unique checkNumber, the paymentDate, and the amount.

```
INSERT INTO payments (customerNumber, checkNumber, paymentDate, amount)
VALUES (103, 'CHK9999', '2026-05-02', 25000);
```